

History of JavaScript

History:- In 1995, A Netscape (browser) programmer named Brendan Eich developed a scripting language in just 10 days.

Originally name (first name):- Mocha

Second name:- LiveScript

At that time java is famous programming language. So, for marketing purpose LiveScript changed into javascript.

★ Java and JavaScript both are different programming language. nothing is common.

Mocha → LiveScript → JavaScript

In 1997, there is another famous browser that was Internet Explorer (Microsoft browser).

Then, Microsoft copied javascript features made own language named as Jscript.

In Browser was (Netscape vs internet explorer)

Netscape → JavaScript

Internet Explorer → Jscript

EcmaScript is born....

Ecma International :- Ecma international is an industry association founded in 1996, dedicated to the standardization of information and communication systems.

JavaScript + Ecma $\rightarrow$ **EcmaScript**.
(Rules)

Problem solved :- We can ^{Same} implement scripting language for different browser.

First EcmaScript.

ES1 $\rightarrow$ 1997

ES5 $\rightarrow$ 2009 (lots of new features)

ES6 (ES2015) $\rightarrow$ 2015 (Biggest update for Js)

ES6 is also known as Modern JavaScript

Ecma have a technical community known as TC39 had decided ~~that~~ after 2015 we release javascript with new featurss every year (**Annual release**).

@programmer-girl--

JavaScript Features.

Features:-

Case sensitive

Dynamically typed

Cross-platform

Interpreted

Object-oriented Scripting language

Backward compatible

@programmer-girl--

JavaScript variables

Variables:- Variables stores the data which can be changed or used when we need.

There are these ***keywords** to declare a variable.

Keywords are the predefined words in programming languages.

- **var** var name = 10;
- **let** let name = 10;
- **const** const pi = 3.14;

Datatype in JavaScript

There are two types of data

1. Primitive
2. Non-primitive.

Primitive datatypes are:-

- Number
- Null
- String
- Bool
- Undefined
- BigInt
- Symbol

@programmer-girl--

Non-Primitive datatypes are:-

- Array
- Object
- RegExp.

JavaScript Hacks

1. Convert string to numbers

Put the pulse (+) before the string

For Example:-

```
let str = '9';  
console.log(typeof(+str));
```

2. Convert number into string

Add a empty string with the numbers

For Example:-

```
let num = 10;  
console.log(typeof(num + ''));
```

@programmer-girl--

JavaScript String.

String:- String are used to store textual form of data like word, sentence. It follows zero based indexing.

```
let str = "pro";
```

```
let str = 'pro';
```

```
let str = `pro`;
```

JavaScript String Method

trim()

slice()

charAt()

toString()

concat()

substring()

indexOf()

toUpperCase()

lastIndexOf()

toLowerCase()

@programmer-girl--

Undefined in JavaScript

- Accessing an uninitialized variable returns undefined.

```
let str;
```

```
console.log(str); //undefined
```

- Accessing a non-existing property of an object returns undefined.
- Accessing a out-of-bounds array element returns undefined

Null in JavaScript

- null means 'no value' assign to variable.
- typeof null returns 'object'
- Null is treated as false value.

@programmer-girl--

JavaScript BigInt

BigInt:- BigInt is a primitive Datatype which is used for large numeric values it doesn't represent decimal values.

It is used to represent values greater than $2^{53}-1$.

Declaration of BigInt

- By appending n at the end of numeric values.

```
var num = 9876543219865252772n;
```

- By passing the values as an argument to the `BigInt()`.

```
var num = BigInt(987654321986525277);
```

@programmer-girl--

JS

Ternary Operator

Ternary operator:- It is also called **conditional operator**.

- It takes three operands.
- It makes the code more concise.

@progammer-girl--

Syntax:-

let variableName = condition ? True : False;

If the condition is true expression after ? will executes. If it is false, expression after : (colon) will executes.

@progammer-girl--

For Example:-

```
let age = 18;  
let warning;  
age >= 18 ? (warning = "You can play")  
           : (warning = "You cannot play");  
console.log(warning);
```

Output:- You can Play.

JS

Boolean Data Type

Boolean :- It can hold only two values:
true and false.

For Example:-

```
Var Read = true;  } typeof(Read) }  
Var Eat = False;   Boolean
```

Boolean values also come as a result of Comparisons.

For Example:-

@programmer-girl--

```
Var x = 1, b = 4, y = 8;
```

```
console.log(b > x) //output:- true
```

```
console.log(b > y) //output:- false
```

== and ===

== (Double equals operator) :- Known as the **Equality** or **abstract** comparison operator.

→ It compare variables, ignores datatype.

=== (Triple equals operator) :- Known as the **identity** or **strict** comparison operator.

→ It compare variables as well datatype.

JS Truthy and Falsy Values

Truthy values:- It is a value that is considered true when encountered in a Boolean context.

Example:-

true, {}, [], 42, "0", "False",
new Date(), -42, 12n, 3.14, -3.14,
Infinity, -Infinity.

@programmes-girl--

Falsy values:- It is a value that is considered false when encountered in a Boolean context.

Example:-

undefined, null, NaN, false, "",
0, -0, 0n (BigInt)

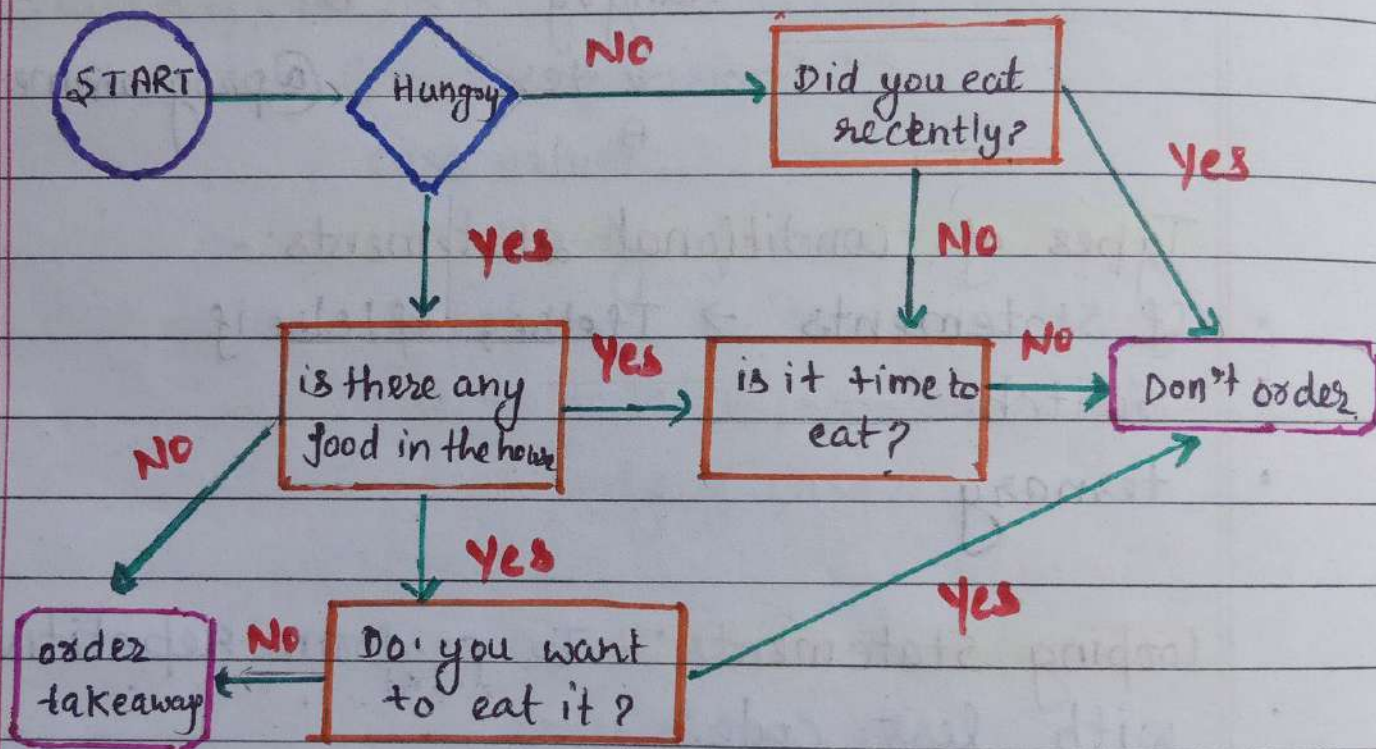
```
var values = 42;  
if (values) {  
    console.log(true);  
}  
else {  
    console.log(false);  
}
```

output:- true

Control Flow

Control Flow:- It allows our program to make decisions about what code is executed and when.

For Example:-



@programmer-girl.

Control Flow have two types of Statements

1. Conditional Statements.
2. Looping Statements.

Conditional statements:- Conditional statements are basically checks to see if a certain condition is either true or false. If the condition is true then run code A, if it's false then run code B.

Hungry $\xrightarrow{\text{NO}}$ B
 $\downarrow$ Yes
 A

@programmer-girl--

Types of conditional statements:-

- If statements $\rightarrow$ If else, if/else if
- Switch
- ternary

Looping Statements:- To perform repetitive task with less code.

Types of loops:-

- for loop
- do/while
- for -- in
- for -- of

Switch Statement

Switch statement:- It evaluates an expression compare its result with case values and execute the statement associated with the matching case.

Switch Syntax:-

```
Switch (expression) {
```

```
    case value1:
```

```
        // body of case 1.
```

```
        break;
```

```
    case value2: @programmer-girl--
```

```
        // body of case 2
```

```
        break;
```

```
    default;
```

```
        // body of default
```

```
}
```

break:- It is optional. It is used to end the switch statement.

Default:- If there is no matching case, the default body executes. It is optional.

For loop

For loop:- For loop executes a block of code as long as a specified condition is true.

Syntax:-

@programmer-girl--

```
for (initializers; condition; iterator) {  
    // statements  
}
```

→ **Initializer:-** It is an expression that initialize the loop, it executed once.

→ **Condition:-** It is a boolean expression that determines whether the for loop should execute or stop.

→ **Iterator:-** For statement executes the iterator after each iteration.

→ **Example:-**

```
for (let i = 2; i < 4; i++) {  
    console.log(i);  
}
```

While, Do While loops

While loop :- While loop executes statements as long as the conditions are true. If the condition become false, the loop is terminated.

Syntax:-

```
while (condition) {  
    // statements  
}
```

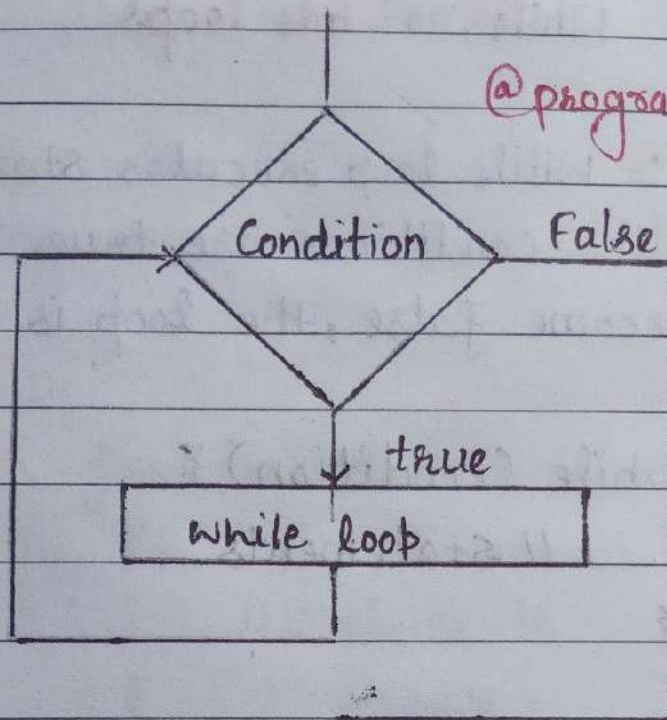
Do while loop:- In Do while loop, the block of code executed once even before checking the condition.

Syntax:-

```
do {  
    // statements  
} while (condition)
```

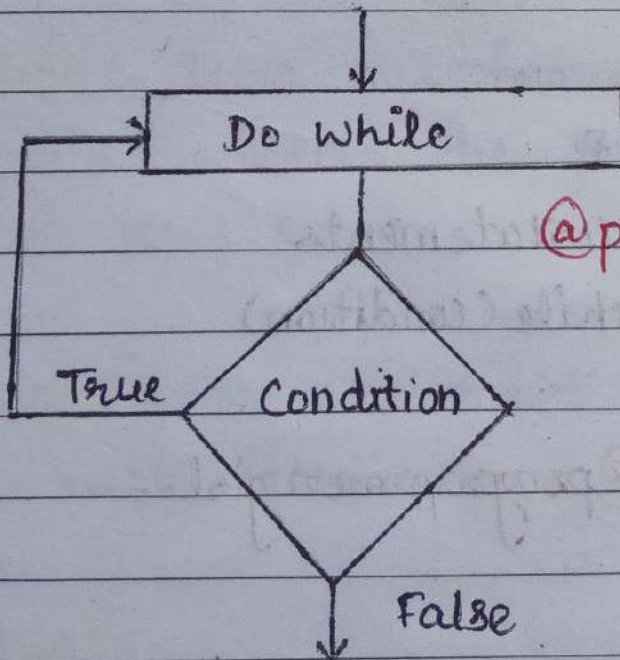
@programmer-girl--

@programmer-girl--



Loop Terminates

∴ **While loop** ∴



@programmer-girl--

Loop Terminates

∴ **Do... While loop** ∴

JavaScript Functions

Function:- A function is a block of code that performs a specific task.

Declare Function:-

```
function funName() {  
    // statements  
}
```

function is declared using the function keyword.

@programmer-girl--

Call Function:-

```
function funName() {  
    // statements  
}
```

funName(); → Call Function.

Example:-

```
function myName() { → Declare Fun.  
    console.log("Smily");  
}
```

myName(); → Function call

Output:- Smily

Advantage of function:-

Reusability

less code

Easy to understand

Function Parameters:- When we declare function we specify the parameters.

Function Arguments:- When we call function we specify the arguments.

For Example:-

```
function example (Parameter) {  
  console.log (Parameter);  
}
```

```
let argument = 'arg';  
example (argument);
```

@programmer-girl--

-: Intro to Arrays :-

Arrays:- It is a ordered collection of items.

Element / item
↓
let pets = ["cat", "dog", "cow"];
Index. → 0 1 2

JavaScript Array Characteristics

1. It can hold values of mixed types.
2. Size of Array is dynamic.

@programmer-girl--

let mixed = [1, 2.5, "cat"]; → Mixed Type.
pets.push("Monkey"); } → Dynamic Size.
console.log(pets);

Accessing Array Elements

Arrays are zero-based indexed. It means the first element of array starts at index zero.

let pets = ["cat", "dog"];
console.log(pets[0]); → Accessing element
output:- cat

-: Array Methods :-

1. **Array Length**:- It returns the number of elements in an array.

```
let num = [1, 2, 3, 4];  
console.log(num.length); // 4.
```

2. **Array Push()**:- It adds elements to the end of the array.

```
let num = [1, 2, 3];  
console.log(num.push(4));  
// [1, 2, 3, 4]
```

3. **Array Pop()**:- It removes the last element from an array and returns removed element.

```
let num = [1, 2, 3, 4]  
let removednum = num.pop();  
console.log(num); // [1, 2, 3]  
console.log(removednum); // 4.
```

@programmer-girl--

4. **Array Shift()**:- It removes the first element and returns removed element from an array.

```
let num = [1, 2, 3, 4];  
let removednum = num.shift();  
console.log(num); // [2, 3, 4]  
console.log(removednum); // 1
```

@programmer-girl--

5. **Array unshift()**:- It adds elements to the beginning of an array.

```
let num = [1, 2, 3, 4];  
console.log(num.unshift(0));  
// [0, 1, 2, 3, 4]
```

6. **Array Sort()**:- It sorts the items of an array.

```
let num = [0, 2, 4, 1];  
console.log(num.sort());  
// [0, 1, 2, 4]
```

7. **Array reverse()**:- It returns the reverse items of an array.

```
let num = [1, 2, 3, 4];  
console.log(num.reverse());  
// [4, 3, 2, 1]
```

Primitive vs Reference Types.

Primitive Types	Reference Types
It has a fixed size in memory.	Do not have a fixed size in memory.
Data Stored on the Stack <i>@programmer-girl--</i> Stored directly in the location.	object stored in the heap. Stored in the variable location is a pointer
For Example:- Null, String, Number, Bool undefined, Symbol	For Example:- Arrays, Objects, Functions, Dates
We cannot add, delete update in primitive data.	We can add, delete update in reference data.

Spread Operator

Spread operator :- It is used to expand or spread an iterable or an array. It is denoted by three dots (...)

For Example :-

```
let arrStr = ['A', 'B', 'C'];  
console.log(arrStr); // ['A', 'B', 'C'];  
console.log(...arrStr); // ABC
```

Clone Array using Spread operator :-

```
let arr1 = [1, 2, 3];  
let arr2 = [...arr1]  
console.log(arr1); // [1, 2, 3]  
console.log(arr2); // [1, 2, 3]  
// append an item to the array.  
arr1.push(4);  
console.log(arr1); // [1, 2, 3, 4]  
console.log(arr2); // [1, 2, 3]
```

@programmer-girl--

Array Destructuring

Array destructuring:- It is used to assign array values to distinct variables.

Example:-

```
const items = ['Books', 'Pen', 'Pencil'];  
const [x, y, z] = items → destructuring  
console.log(x); // Books  
console.log(y); // Pen  
console.log(z); // Pencil
```

@programmer-girl--

Destructuring by using spread operator

```
const [x, ...y] = items  
console.log(x) // Books  
console.log(y) // ['Pen', 'Pencil']
```

Note:- We should use variable with spread syntax as the last variable otherwise it throw error

I don't like error! Do you...

```
const [...y, x] // error
```

Objects Introduction

Objects:-

- They are reference type
- objects are good to handle real world data
- objects stores data in key value pairs.
- objects don't have index.

@programmer-girl--

Object declaration:-

```
const person = {
```

```
  key ← name: 'coder', → value
```

```
  key ← age: 20 → value  
};
```

```
console.log(typeof person); // object
```

→ Access data from objects

```
console.log(person.name); // coder
```

```
console.log(person.age); // 20
```

↘ dot notation

→ Add key-value pairs to objects

```
person.id = 5;
```

```
console.log(person);
```

```
// { name: 'coder', age: 20, id: 5 }
```

Another Method :- Bracket Notation

```
const person = {  
  name: 'codes';  
  "person age": 22;  
};
```

Key stored as a string by default.

Let's access data by Bracket Notation

```
console.log(person["name"]);  
// output :- codes
```

Bracket Notation vs Dot Notation

In above example there is a key named as "person age" let's access it by dot Notation

Dot Notation:- `console.log(person.person age);`
↓
It gives error because Js not include spaces between names.

Bracket Notation:- `console.log(person["person age"]);`
↓
It works because it becomes string now.

@programmer-girl--

Iterate Object

∴ Using for...in loop

```
let person = {
```

```
  firstname: 'programmer',
```

```
  last name: 'girl',
```

```
  age: 21
```

@programmer-girl--

```
};
```

```
for (let key in person) {
```

→ Console.log(key); // output:-
// It access //
// only key // age

→ Console.log(person[key]); // output:-
// It access //
// only values // programmer
// girl
// 21

→ Console.log(key, ":", person[key]);
It access // output:-
both key-value. //
//
age: 21

Object.keys() :- This method was introduced in ES6. It takes an object and returns an array of the object properties. (Key)

For Example:-

```
console.log(Object.keys(person));
```

// output:-

```
["firstname", "lastname", "age"]
```

Object.values() :- It takes an object and returns an array of the object values.

For Example:-

@programmer-girl--

```
console.log(Object.values(person));
```

// output:-

```
["programmer", "girl", "21"]
```

Object.entries() :- It takes an object and returns the key-value pair.

For Example:-

```
console.log(Object.entries(person));
```

// output:- Try yourself--

Object Destructuring

Object destructuring:- It assigns properties of an object to individual variables.

Example:-

```
let person = {  
  name: 'Coder',  
  age: 'twenty'  
};  
@programmer-girl--
```

```
let name = person.name;
```

```
let age = person.age;
```

We typically do like ↗

```
↘ let { name, age } = person;
```

object console.log(name); // 'Coder'

destructuring console.log(age); // 'twenty'

Setting default values:-

```
let { name, age, class = '' } = person
```

```
console.log(class); // ''
```

No class property in person object, Then we assign an empty string to the class.

Arrow Functions

Arrow Function:- Another way to write a function.
It is Introduced in the ES6 version of JS.
It's Syntax is shorter than regular Function.

Example:-

→ Function Expression
let add = function(a,b) {
 return a+b;
};

Above code using arrow function

```
let add = (x,y) => {  
    return x+y;  
};
```

Function with single parameters

(p1) => { statements } // Syntax -1
p1 => { statements } // Syntax -2.

Function with no parameter

```
let a = () => {    // Syntax.  
    return 0  
};
```

@programmer-girl--

Hoisting

Hoisting:- It is a behavior in which a function or a variable can be used before declaration.

Variable hoisting:-

```
console.log(name); // undefined  
var name = "xyz";
```

It doesn't cause an error. Because it looks like in execution phase:-

```
var name; @programmer-girl--  
console.log(name);  
var name = "xyz";
```

In case of **let keyword**:-

```
console.log(name); // Reference Error  
let name = "xyz";
```

It cause an error. In case of let, variable is ~~not~~ hoisted but not initialized.

In case of **const keyword**:-

```
console.log(name); // Error  
const name = "xyz";
```

Conclusion:- let and const variables are hoisted but they cannot be accessed before their declaration.

Function Hoisting:- Function can be called before declaring it.

name(); → Function called

Declaration ← function name() {
 console.log('programmer-girl--');
} → **Formal function**

output:- programmer-girl--

Function Expression:- TypeError occurs in case of function expression.

name(); → **Function Expression.**

```
var name = function() {  
    console.log('programmer-girl--');  
}
```

Arrow Function:-

name(); // **Type Error**

```
var name = () => {
```

```
    console.log('programmer-girl--');  
}
```

Conclusion:- JavaScript doesn't hoist the function expressions and arrow functions.

@programmer-girl--

Lexical Scope

lexical scope:- It means that a variable defined outside a function can be accessible inside another function defined after the variable declaration. But the opposite is not true.

For Example:-

```
function add() {  
    var x = 4; // y is not accessible  
    function mul() {  
        // x is accessible here, y is not  
        function minus() {  
            var y = 6; // x is accessible.  
        }  
    }  
}
```

@programmer-girl--

Note:- The variable defined inside a function will not be accessible outside the function. In above code y is not accessible outside the function.

Block Scope vs Function Scope

Block Scope	Function Scope
It means that the variable is accessible within the block <code>{ }</code>. @programmer-girl-- let and const are block scope. For Example:- <pre>for (let i=0; i<10; i++){ // Block }</pre> <code>console.log(i);</code> // Error we are trying to access let variable outside the scope.	It means that the variables are only accessible in the function in which they are declared. var is a function scope. For Example:- <pre>function fun() { var x = 42; }</pre> <code>fun();</code> <code>console.log(x);</code> // Error we cannot access var outside the function scope.

Default Parameters

Default Parameters:- It allows us to give default values to the function parameters if no values is given.

For Example:-

Default parameters
↗ ↘

```
function add(x=1, y=2){  
  return x+y;  
}
```

console.log(add(3,4)); // 7

console.log(add(5)); // x=5, y=2 ⇒ 7

console.log(add()); // x=1, y=2 ⇒ 3

Note:- A parameter has a default value is undefined.

For Example:-

```
function add(x){  
  console.log(x);  
}
```

add(); // output:- undefined

@programmer-girl--

Rest Parameters:

Rest Parameters:- It is used to gather parameters and put them all in an array.

Let's understand with an example:-

```
function test(a,b){  
  console.log(a); 118 → output  
  console.log(b); 119 → output  
}
```

```
test(8,9);
```

```
test(8,9,7,6,5);
```

What will happen with 7,6,5? oops Nothing.
To cover 7,6,5 (Rest parameters concept comes)

```
function test(a, ...b){  
  console.log(a);  
  console.log(b);  
}
```

Rest parameter
Syntax.

```
test(8,9,7,6,5);
```

11 output:- 8

[9,7,6,5]

Hope you understand...

Parameter Destructuring

Parameters destructuring:- A couple of specific property values to pass as an parameter to the function definition, not in entire object.

For Example:-

```
const user = {  
  'name': 'Alex',  
  'age': '20'  
}
```

Param.

→ destructuring

```
function userDetails ({name, age})  
{
```

```
  console.log(name);
```

```
  console.log(age);  
}
```

```
userDetails();
```

→ user

Output:-

Alex

20

@programmer-girl--

CallBack Function

callback function:- It is defined as, we can also pass a function as an argument to a function.

For Example:-

```
function Second(name){  
  console.log(name);  
function first(callback){  
  callback('Alex');  
}
```

first(second); // output:- Alex.

Second function that is passed as an argument inside **first** function is called a callback function.

Note:- The callback function is helpful when you have to wait for a result that takes time.

@programmer-girl--

Sets in JavaScript

Sets :-

Stores a collection of unique values
No index-based access
Order is not guaranteed
Sets are iterable
Sets have its own methods.

Sets Syntax :-

```
let items = new Set();
```

typeof Sets :- Object

@programmer-girl--

instance of Sets :- True

Array Vs Sets :-

- Array can have duplicate values whereas Sets cannot.
- In Array data is ordered by index whereas Sets cannot

Useful Set methods.

`const items = new Set();`

- **add()** :- Append a new element to the end of the set.

For Example :- `items.add('Hi');`

// output :- `Set(1) { 'Hi' }`

- **clear()** :- Remove all the elements and returns undefined.

For Example :- `const items = new Set([1, 2, 3]);`
`items.clear();`

// output :- `Set(0) { }`

- **delete()** :- It delete a specific element from Set.

For Example :- `const items = new Set([1, 2, 3, 4]);`
`items.delete(3);`

// output :- `Set(3) { 1, 2, 4 }`

- **has()** :- check whether an element exists in Sets or not.

`items.has(2);` // output :- `true`.

map data structure

Map:- A Map is a collection of key-value pairs, similar to an object.

Map Syntax:-

```
const person = new Map();
```

★ **Why we need map ? If we have object**

A Map is similar to object, keys in objects are only strings and symbols. But we can use any value as key in Map.

Let's create a map:-

```
const person = new Map();  
person.set('Name', 'Alex');  
console.log(person);
```

// output:- { "Name" => "Alex" }

@programmer-girl--

Methods in Maps

1. **get()**:- Returned the value associated with the key.
2. **set()**:- Set the value of the key and returns the map.
3. **delete()**:- Delete the entry which has the key same as passed key.
4. **clear()**:- Delete all key-value pairs from the map.
5. **has()**:- Returns true if the map has the key provided.
6. **keys()**:- Returns the new iterator that contains the keys insertion order.

@programmer-girl-

Create Own Methods

Methods:- Function inside object is known as methods.

For Example:-

```
const person = {  
  name: 'Alex',  
  age: 21,
```

Method ← `about: function() {
 console.log("My name is
 ${this.name}");`

```
  }  
}  
person.about();
```

Output: My name is alex

When declaring a new object, use the object literal, not the `new object()` constructor.

@programmer-girl--

This keyword

this keyword:- "this" keyword refers to an object that is executing the current piece of code.

For Example:-

```
function Info() {  
  console.log('my name is $this.  
  name');
```

```
}
```

```
const person1 = {  
  name: 'Codes',  
  about: Info
```

```
} @programmer-girl--
```

```
const person2 = {  
  name: 'girl',  
  about: Info
```

```
}
```

person1.about(); → person1 name

person2.about(); → person2 name.

// output:- my name is Codes

my name is girl.

Call, Apply, Bind Methods

- **call**:- It invokes the function and allows you to pass in arguments one by one.
- **Apply**:- It invokes the function and allows you to pass in arguments as an array.
- **Bind**:- It returns a new function, allowing you to pass in a this array and any no. of arguments.

* **Call Example**:-

```
const user1 = {  
  name: 'Alex',  
  age: 21, @programmer-girl--  
  intro: function() {  
    console.log(this.name, this.age);  
  }  
}
```

```
}
```

```
const user2 = {
```

```
  name: 'Jhon', // output:-
```

```
  age: 31 } } ↗ Jhon, 31
```

```
user1.intro.call(user2);
```

★ Apply Example :-

```
var user1 = { name: 'Alex',  
              age: 21 };
```

```
var user2 = { name: 'Jhon',  
              age: 31 };
```

```
function say(greet) {  
  console.log(greet + this.name);  
}
```

```
say.apply(user2, ['Hi']);
```

// output:- HiJhon

★ Bind Example :-

```
var user1 = { name: 'Alex',  
              age: 21 };
```

```
var user2 = { name: 'Jhon',  
              age: 31 };
```

```
function say() {  
  console.log(this.name, this.age);  
}
```

```
var myfun = say.bind(user1);  
myfun();
```

// output:- Alex 21

@programmer-girl--

Prototype

Prototype:- It is used to add new properties and methods to an existing object constructor.

@programmer-girl--

For Example:-

```
function Person() {  
  this.name = 'John',  
  this.age = 23  
}
```

```
const person = new Person();
```

```
console.log(Person.prototype);
```

↳ checking the prototype value

// output:- {....}

It shows an empty object.

Prototype Inheritance:- Objects inherit properties and methods from a prototype. Using the prototype makes faster object creation since properties / methods on the prototype don't have to be re-created each time a new object is created.

new keyword

Five things to remember about new keyword

1. It creates a new object. The type of this object is simply object
2. It sets this new object's internal.
3. It makes the **this** variable point to the newly created object
4. It executes the constructor function, using the newly created object whenever **this** is mentioned.
5. It returns the newly created object.

@programmer-girl--

Thank You

Keep learning!

Keep coding!

Keep Smiling :)

How do you feel about notes

-: tell us :-

@programmer-girl--